

Assignment-4
Parallel and Distributed Computing
Submitted by:
Aunish Kumar Yadav (102303306)

Q1: Ring Communication

Objective: Write an MPI program where each process sends a message to the next process in a ring topology.

Performance Data:

Processes (p)	Time (Tp) in sec	Speedup	Efficiency	Comm %
1	< 0.000001	N/A	N/A	0.00%
2	0.000141	~0.01	~0.50%	100.00%
4	0.000143	~0.01	~0.25%	100.00%

Execution Output:

Process 0 started with value: 100
Process 1 received value: 100
Process 2 received value: 101
Process 3 received value: 103
Process 0 received final wrapped value: 106

Analysis & Key Inferences:

- **Inference 1: Latency-Dominated Execution**
For a single process, execution time is negligible since it only performs local computation. For multiple processes, total time reflects communication latency. The increase in time with more processes shows that ring communication follows $O(p)$ behavior due to sequential message passing.
- **Inference 2: Communication Overhead Dominance**
The computation per process is minimal (only adding rank), so nearly all execution time is spent in MPI_Send and MPI_Recv operations. This results in communication overhead dominating performance.

- **Inference 3: No Parallel Speedup**

There is no parallel computation in this program. Each process must wait for data from the previous one, making the execution inherently sequential despite multiple processes.

- **Inference 4: Increasing Processes Increases Delay**

As the number of processes increases, total execution time increases due to added communication steps and synchronization delays. This demonstrates that ring topology is not suitable for performance scaling but for communication pattern study.

Q2: Parallel Array Sum

Objective: Compute the sum of an array of size 100 in parallel using MPI_Scatter and MPI_Reduce.

Performance Data:

Processes (p)	Time (Tp) in sec	Speedup (Sp)	Efficiency (Ep)	Comm %
1	0.000006	1.00	100.00%	0.00%
2	0.000828	0.01	0.50%	99.27%
4	0.000146	0.04	1.03%	95.89%

Execution Output:

Global Sum: 5050 (Expected: 5050)

Average Value: 50.50

Analysis & Key Inferences:

- **Inference 1: Parallelization Overhead Dominates**

For a small array (size = 100), sequential execution is extremely fast. Introducing MPI adds communication overhead (scatter and reduce), which increases total execution time instead of reducing it.

- **Inference 2: Poor Speedup and Efficiency**

Speedup remains close to zero and efficiency is very low across all process counts. This indicates that the cost of communication is much higher than the computation being performed.

- **Inference 3: Optimal Performance at Low Process Count**

Among parallel runs, P=4 performs better than P=2 and P=8, but still worse than P=1. This shows that increasing processes beyond a certain point leads to diminishing returns due to overhead.

- **Inference 4: High Communication Cost**

Communication percentage is above 90% for all parallel cases, proving that most of the execution time is spent in MPI_Scatter and MPI_Reduce rather than actual computation.

Q3: Finding Maximum and Minimum

Objective: Generate 10 random numbers per process and find the global maximum and minimum values using MPI_MAXLOC and MPI_MINLOC.

Performance Data:

Processes (p)	Time (Tp) in sec	Speedup (Sp)	Efficiency (Ep)	Comm %
1	0.000003	1.00	100.00%	0.00%
2	0.000144	0.02	1.04%	97.91%
4	0.004108	0.00	0.02%	~100.00%

Execution Output:

Process 0 generated: 543 941 64 194 254 12 26 499 575 723

Process 1 generated: 405 975 861 261 476 433 208 960 407 182

Process 2 generated: 400 667 778 190 675 194 857 929 876 570

Process 3 generated: 839 181 901 944 285 839 452 867 283 17

--- Global Results ---

Global Maximum: 975 (Found in Process 1)

Global Minimum: 12 (Found in Process 0)

Analysis & Key Inferences:

- **Inference 1: Communication Overhead Dominates**

The computation per process (finding local max/min) is very small, while MPI reduction introduces significant overhead, leading to very high communication cost (>97%).

- **Inference 2: Performance Degradation with More Processes**

Execution time increases drastically from P=2 to P=4. This shows that adding

more processes increases synchronization and communication delays instead of improving performance.

- **Inference 3: Poor Speedup and Efficiency**

Speedup is close to zero and efficiency drops sharply as processes increase. This indicates that the parallelization is not beneficial for such a small workload.

- **Inference 4: Latency-Sensitive Operation**

Using MPI reduction with value-rank tracking (e.g., struct or pair) adds extra data transfer overhead compared to simple scalar reductions, further increasing latency.

Q4: Parallel Dot Product

Objective: Compute the dot product of two vectors of size 8 in parallel.

Performance Data:

Processes (p)	Time (Tp) in sec	Speedup (Sp)	Efficiency (Ep)	Comm %
1	0.000022	1.00	100.00%	0.00%
2	0.000071	0.31	15.29%	84.51%
4	0.000161	0.14	3.41%	96.58%

Execution Output:

Parallel Dot Product Result: 120

Expected Result: 120

Analysis & Key Inferences:

- **Inference 1: Extremely Small Workload**

The vectors are very small (N=8). At higher process counts, each process performs only a few multiplications, making computation negligible compared to communication.

- **Inference 2: Negative Scaling Effect**

Execution time increases as processes increase (from P=2 to P=4). This shows

negative scaling, where adding more processes degrades performance due to overhead.

- **Inference 3: Communication Dominates Execution**

A large portion of time (>80%) is spent in MPI_Scatter and MPI_Reduce. The actual computation contributes very little to total runtime.

- **Inference 4: Low Efficiency at Higher Processes**

Efficiency drops significantly to around 3% at P=4, indicating poor utilization of parallel resources.

- **System Design Takeaway:**

MPI-based parallelization is not suitable for very small datasets. The communication overhead outweighs computation, making sequential execution faster and more efficient.